

Syrian Arab Republic

Ministry of Higher Education

Syrian Virtual University



Bachelor of IT Program BAIT

MS SQL Server Administration IIS404

Chapter Two

Backup & Restore in MS SQL Server

Eng. Ayham Mohammad

Contents

1.	Intro	1
2.	Why backup?	1
3.	Backup Types.....	1
3.1.	Copy-only backup.....	2
3.2.	Data backup	2
3.2.1.	Database backup	2
3.2.2.	File backup	2
3.2.3.	Partial backup.	2
3.2.4.	Full backup.....	3
3.2.5.	Differential backup	3
3.2.6.	Log backup.	3
4.	Backup Compression.....	4
5.	Recovery Models	5
5.1.	Simple	5
5.2.	Full	5
5.3.	Bulk logged	5
6.	Restore and Recovery.....	6
6.1.	Overview of Restore Scenarios	6
6.2.	Steps to restore a database.....	7
6.2.1.	Advantages of a File or Page restore	8
6.2.2.	Recovery and the transaction log.....	8
6.3.	Accelerated database recovery	9
7.	Restore Scenarios – Full Recovery Model	9
7.1.	To Restore to point P6:	9
7.2.	To Restore to a point after latest backup:.....	10
7.3.	Restore to a point of time	11
7.4.	Piecemeal Restore	11
7.4.1.	Piecemeal Restore Scenarios.....	12

7.4.2.	Piecemeal Restore of Database (Simple Recovery Model)	12
7.5.	Piecemeal Restore of Database (Full Recovery Model)	13
7.5.1.	Restore Sequences	13
7.6.	Restore Pages.....	14
7.6.1.	Limitations	15
7.6.2.	Page restore scenarios:	16

1. Intro

The SQL Server backup and restore component provides an essential safeguard for protecting critical data stored in your SQL Server databases. To minimize the risk of catastrophic data loss, you need to back up your databases to preserve modifications to your data on a regular basis.

A well-planned backup and restore strategy helps protect databases against data loss caused by a variety of failures. Test your strategy by restoring a set of backups and then recovering your database to prepare you to respond effectively to a disaster.

2. Why backup?

Backing up is the way to protect your data, with valid backups, you can recover your data from many failures, such as:

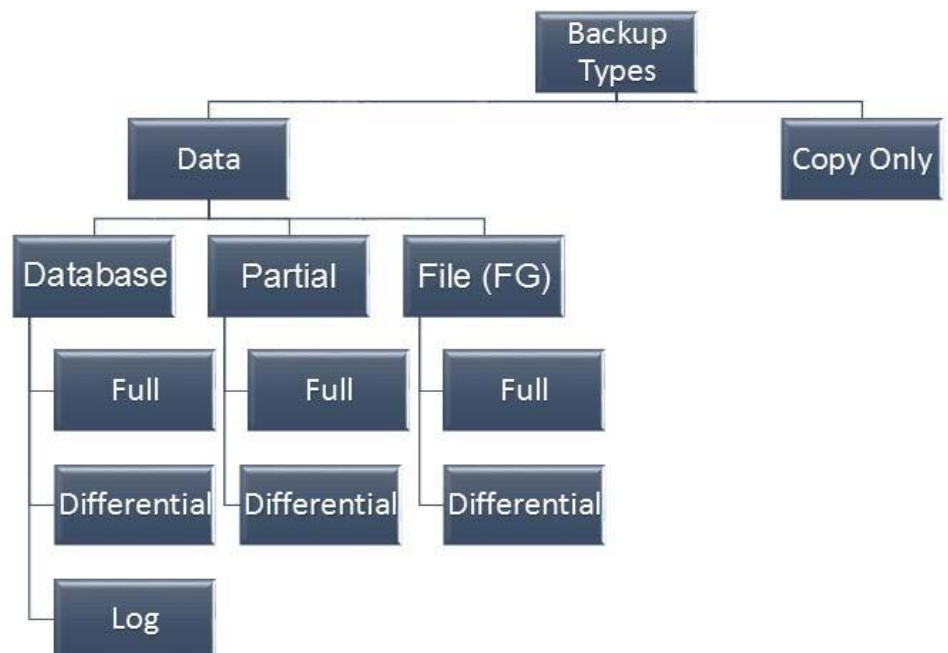
- Media failure.
- User errors, for example, dropping a table by mistake.
- Hardware failures, for example, a damaged disk drive or permanent loss of a server.
- Natural disasters. By using SQL Server Backup to Windows Azure Blob storage service, you can create an off-site backup in a different region than your on-premises location, to use in the event of a natural disaster affecting your on-premises location.

*عملية اخذ نسخة احتياطية من قاعدة البيانات هي Backup

عملية استعادة هذه النسخة هي Restore

عملية استعادة البيانات الضائعة بين لحظة اخذ الباك اب ولحظة انهيار قاعدة البيانات هي Recovery

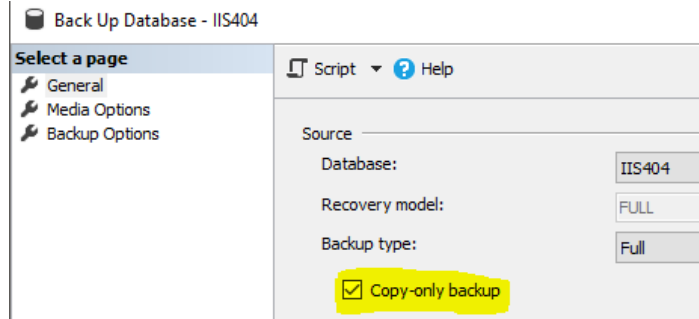
3. Backup Types



3.1. Copy-only backup

A special-use backup that is independent of the regular sequence of SQL Server backups.

نسخة احتياطية لاستخدام خاص مستقل عن التسلسل العادي للنسخ الاحتياطي



أو بالتعلمية: backup database DB-name to disk = 'path of backup file.bak' with copy_only

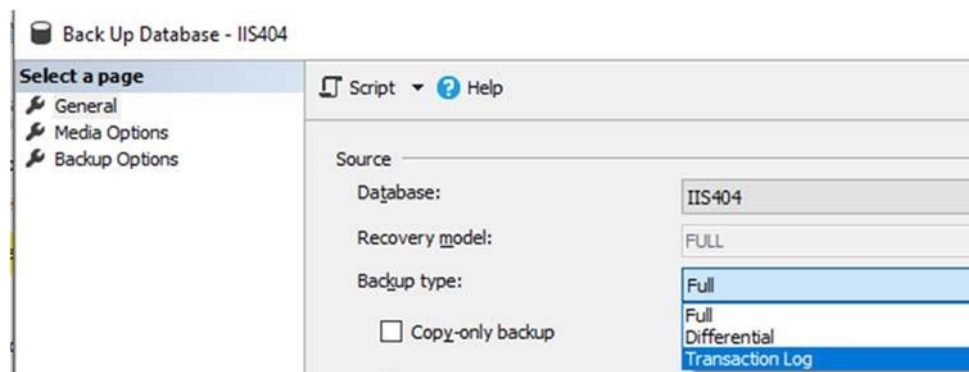
3.2. Data backup

A backup of data in a complete database (a database backup), a partial database (a partial backup), or a set of data files or filegroups (a file backup).

3.2.1. Database backup

A backup of a database. Full database backups represent the whole database at the time the backup finished. Differential database backups contain only changes made to the database since its most recent full database backup.

باستخدام ال GUI:



أو بالتعلمية: backup database DB-name to disk = 'path of backup file.bak'
و لإستعادتها بالتعلمية: restore database DB-name from disk = 'path of backup file.bak'
نستطيع إستخدام نفس التعلمية للإستعاضة عن إنشاء DB جديدة في حال لدينا نسخة Backup عنها من شخص ما
و في حال الحصول على نسخة معذلة من ال DB التي لدي، و عدم إرادتي بتضييع النسخة التي لدي. نستطيع القيام ب restore لها مع تعديل إسمها فقط – حيث
سيتم حفظها بالإسم الجديد الذي قمنا بإختياره و لن يتم التعديل على أي من أسماء ملفاتنا

أو نقوم بالتعديل على إسمها و مسار حفظها في حال إرادتنا إنشاء نسخة مطابقة عنها لدينا بغير إسم باستخدام:

```
restore database new-DB-name from disk = 'path of backup file.bak'  
with
```

```
Move 'file1name' to 'new-path.mdf'
```

```
Move 'file2name' to 'new-path.ldf'
```

3.2.2. File backup

Filegroups نسخة احتياطية من واحد أو أكثر من ملفات قاعدة البيانات أو

```
backup database [DB-name] filegroup = 'file group name' to disk = 'path of backup file.bak' أو بالتعلمية:
```

3.2.3. Partial backup

النسخ الاحتياطية الجزئية مفيدة عندما تريد استبعاد مجموعات الملفات المخصصة للقراءة فقط

```
backup database DB-name name-of-file-groups to disk = 'path of backup file.bak' أو بالتعلمية:
```

```
Backup database iis404 Read_Write_Filegroups To Disk= 'D:\iis404Partial.bak'
```

3.2.4. Full backup

نسخة احتياطية للبيانات تحتوي على جميع البيانات في قاعدة بيانات محددة أو مجموعة من الملفات أو الملفات و دائما يجب البدء بها و يرمز لها **F**

3.2.5. Differential backup

يحتوي ال Data منذ آخر Full Backup إلى هذه اللحظة حتى لو وجد نسخ Differential Backup قبله, و يتم بها حفظ ال Extents التي تغيرت فقط منذ آخر عملية Full Backup و يرمز لها **D**

3.2.6. Log backup

يرمز لها ب **L** و يحتوي ال Data منذ آخر عملية Backup مهما كان نوعها

Minimally, you must have created at least one full backup before you can create any log backups. After that, the transaction log can be backed up at any time unless the log is already being backed up. It's recommended to take log backups frequently, both to minimize work loss exposure and to truncate the transaction log. If a database is damaged or you are about to restore the database, we recommend that you create a tail-log backup to enable you to restore the database to the current point in time.

Tail-Log Backup contains committed transaction log data that has not been backed up.

و يرمز لها ب **TLB** يحوي فقط المناقلات التي تم عمل Commit لها (المناقلات في حالة ال Active لا يتم عمل Backup لها) منذ آخر عملية Backup و يترك ال DB في وضع Restoring Mode عند اختيار خيار ال With NoRecovery

The appropriate frequency for taking log backups depends on your tolerance for work-loss exposure balanced by how many log backups you can store, manage, and, potentially, restore. Think about the required RTO and RPO when implementing your recovery strategy, and specifically the log backup cadence.

Recovery Point Objective (RPO), indicates how much data loss is acceptable in the case of disaster.

Recovery Time Objective (RTO), which defines the acceptable downtime for the recovery process.

Tail-Log Backup

It's recommend to take a tail-log backup in the following scenarios:

جميع أنواع ال Backup تضع ال DB في وضع Offline أثناء عملية ال Backup

و عند الإنتهاء منها تكون عودة ال DB لوضع Offline/online اختيارية , حيث عند اختيار With NoRecovery لن تعود ال DB لوضع Online وسيتم وضعها في Restoring Mode والتي يساوي حالة ال Read-Only كما يتم عند استخدام ال L, T, Backups و عند القيام ب Backup لعدة نسخ متسلسلة مثل F1, D1, L1, T نقوم باستخدام خيار ال With Norecovery مع ال F1, D1, L1 و عندما نصل لل T نستخدم خيار ال With Recovery لكي تعود ال DB لوضع ال Online و يستطيع ال Users استخدامها و عند اختيار Continue_After_Error ستعود لوضع ال Online و يتم استخدامه عندما تكون ال DB Damaged

مثال توضيحي

D= Differential backup

F=Full backup

F, D, D₂, L, L₂

L= Log backup

إذا بدى استرجع بعد d2 فيكون الاستعادة f1 d2 لانو d2 بتحتوي d1
بدى استرجع بعد l2 فيكون الاستعادة f1 d2 L1 L2 لانو L2 مابتحتوي L1

اللاحقة لل Backup تكون bak.

نستطيع تغيير اسم ال DB بتحديثها , و لكن ملفاتها لن يتغير اسمهم

- If the database is online and you plan to perform a restore operation on the database, begin by backing up the tail of the log. To avoid an error for an online database, you must use the **WITH NORECOVERY** option of the BACKUP Transact-SQL statement.
- If a database is offline and fails to start and you need to restore the database, first back up the tail of the log. Because no transactions can occur at this time, using the WITH NORECOVERY is optional.

في حال كانت ال DB في وضع Offline يصبح إستعمال الخيار With Norecovery إختياري (لأن ال DB بكل الأحوال في وضع Offilne)

في حال كانت ال DB في وضع Damaged, يجب إستعمال الخيار Continue_after_error عوضا عن خيار With Norecovery لإكمال العملية بحال حدوث أخطاء

- If a database is damaged, try to take a tail-log backup by using the **WITH CONTINUE_AFTER_ERROR** option of the BACKUP statement.
- On a damaged database backing up the tail of the log can succeed only if:
 - the log files are undamaged, the database is in a state that supports tail-log backups, and the database doesnot contain any bulk-logged changes.

شروط نجاح ال Tail-Logged Backup في حال ال Damaged DB:

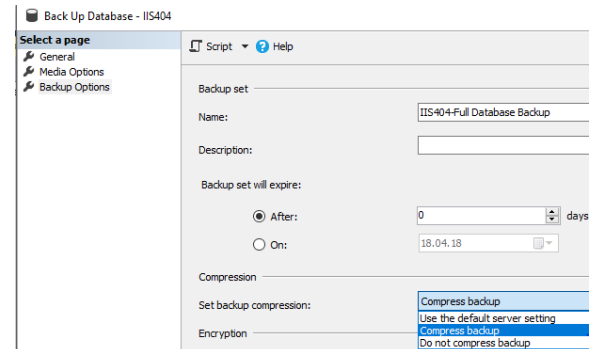
مثل ال Bulk-insert مثلا و هو Insert بيانات كبيرة من ملف مثلما يحدث في نظام تسجيل الدوام

لذلك ينصح بالقيام ب Tail-log-Backup بعد كل Bulk-Insert

- If a tail-log backup cannot be created, any transactions committed after the latest log backup are lost.

The following table summarizes the BACKUP NORECOVERY and CONTINUE_AFTER_ERROR options:

BACKUP LOG option	Comments
NORECOVERY	Use NORECOVERY whenever you intend to continue with a restore operation on the database. NORECOVERY takes the database into the restoring state. This guarantees that the database does not change after the tail-log backup. The log will be truncated unless the NO_TRUNCATE option or COPY_ONLY option is also specified. ال Truncate حذف ال log القديم Important: Avoid using NO_TRUNCATE, except when the database is damaged.

CONTINUE_AFTER_ERRORUse CONTINUE_AFTER_ERROR only if you are backing up the tail of a **damaged database**.

4. Backup Compression

Compressed backup is smaller than an uncompressed one of the same data, compressing a backup typically requires less device I/O and therefore usually increases backup speed significantly.

نستخدم المثال التالي : (1) ننشأ DB جديدة:

Create database svu on

(name='svu_data', filename='path.mdf')

Log on

(name='svu_log', filename='path.ldf')

Go

Use svu

Go

(2) ننشأ جدول جديد في قاعدة البيانات:

Drop table if exists table1

Create table table1

([time of transaction] time,[backup type] varchar(2))

Go

(3) ننشأ Full Backup : F1 -

Insert into table1 vlaues(getdate(), 'F1')

Backup database svu to disk='path\F1.bak'

Select * from table1

(4) ننشأ Differential Backup : D1 -

Insert into table1 values(getdate(), 'D1')

Backup database svu to disk='path\D1.bak' with differential

Select * from table1

(5) ننشأ Differential Backup : D2 -

Insert into table1 values(getdate(), 'D2')

Backup database svu to disk='path\D2.bak' with differential

Select * from table1

(6) ننشأ Trans. Log Backup : L1 -

Insert into table1 values(getdate(), 'L1')

Backup log database svu to disk='path\L1.bak' with differential

Select * from table1

(7) ننشأ Trans. Log Backup : L2 -

Insert into table1 values(getdate(), 'L2')

Backup database svu to disk='path\L2.bak' with differential

Select * from table1

Another Entry after the latest Backup

Insert into table1 values(getdate(), 'T')

Select * from table1

Now we want to restore everything....

Remember that backup were F1 D1 D2 L1 L2

ننتقل إلى أي DB أخرى لكي نستطيع القيام ب Restore لل SVU DB و إلا لن
يقوم بالتنفيذ:

Use mater

ننشأ Tail Log Backup لحفظ آخر عملية insert قمنا بها بعد ال L2 Backup
ثم نبدأ بعملية ال Restore:

Backup log svu to disk='path\T1.bak' with norecovery

إن أردنا استخدام ال SVU DB الآن سيعطينا خطأ، لأننا وضعناها بآخر تعليمة في
وضع ال Restoring Mode بسبب خيار ال Norecovery

Restore database svu from disk='path\F1.bak' with norecovery

Restore database svu from disk='path\D1.bak' with norecovery

Restore database svu from disk='path\D2.bak' with norecovery

Restore database svu from disk='path\L1.bak' with norecovery

Restore database svu from disk='path\L2.bak' with norecovery

بالنسبة لآخر عملية Restore نستطيع أن نستخدم معها خيار ال NoRecovery و
من ثم نعيد ال DB لوضع ال Online يدويا كما سنفعل الآن، أو في آخر تعليمة
Restore لا نستخدم خيار ال NoRecovery و تعود ال DB لوضع ال Online
لوحدها

Restore database svu from disk='path\T1.bak' with norecovery

Restore database svu with recovery

للتأكد من صحة عملنا، نتحقق:

Use svy

Select * from table1

5. Recovery Models

5.1. Simple

- No log backups.

يتم الرجوع لأخر Backup من أي نوع ما عدا ال T, و طريقة الكتابة تجعلها غير كبيرة الحجم و سريعة (تستخدم مع ملفات غير مهمة جدا)

- Automatically reclaims log space to keep space requirements small, essentially eliminating the need to manage the transaction log space.
- Operations that require transaction log backups are not supported by the simple recovery model. The following features **cannot be used** in simple recovery mode:
 - Log shipping
 - Always On or Database mirroring
 - Media recovery without data loss
 - Point-in-time restores

• Work loss exposure

- Changes since the most recent backup are unprotected. In the event of a disaster, those changes must be redone.
- Recover to point in time?
 - Can recover only to the end of a backup

5.2. Full

- Requires log backups.

- No work is lost due to a lost or damaged data file.

- Can recover to an arbitrary point in time (example, prior to application or user error)

• Work loss exposure

- Normally none.
- If the tail of the log is damaged, changes since the most recent log backup must be redone.
- Recover to point in time?
 - Can recover to a specific point in time, assuming that your backups are complete up to that point in time.

5.3. Bulk logged

- Requires log backups.

تعود لأخر حالة Backup حتى لو كانت T, بشرط عدم وجود Bulk-Insert قبلها, حيث إذا كان في ستنمكن من الإستعادة من ال Backup التي قبل ال T مباشرة

حيث تحتفظ بجميع البيانات التي تم كتابتها في ال Transaction Log File عدا العمليات الناتجة عن ال Bulk-Operation

- ال Bulk-insert هو Insert بيانات كبيرة من ملف (ملف .txt أو .csv. مثلا) مثلما يحدث في نظام تسجيل الدوام

لذلك ينصح بالقيام ب Tail-log-Backup بعد كل Bulk-Insert

- An adjunct of the full recovery model that permits high-performance bulk copy operations.
- Reduces log space usage by using minimal logging for most bulk operations
- Work loss exposure
 - If the log is damaged or bulk-logged operations occurred since the most recent log backup, changes since that last backup must be redone.
 - Otherwise, no work is lost.
- Recover to point in time?
 - Can not recover to a point in time if there are any bulk logged operations in the transaction log to restore from.

في ال File, Partial, Full Backups يتم Truncate لل log بعكس ال Copy Only

6. Restore and Recovery

To recover a SQL Server database from a failure, a database administrator has to restore a set of SQL Server backups in a logically correct and meaningful restore sequence. SQL Server restore and recovery supports restoring data from backups of a whole database, a data file, or a data page, as follows:

The database (a complete database restore)

يتم إستعادة كل شئ و يتم وضع ال DB في وضع Offline

The whole database is restored and recovered, and the database is offline for the duration of the restore and recovery operations.

The data file (a file restore)

يتم إستعادة الملف أو مجموعة الملفات المحددة و تعتبر إحترافية أكثر كونها توفر وقت وفق معيار ال RTO,

مع الملاحظة أن الملف بالإضافة لل File Group التي ينتمي لها سيتم وضعهم في وضع Offline

A data file or a set of files is restored and recovered. During a file restore, the filegroups that contain the files are automatically offline for the duration of the restore. Any attempt to access an offline filegroup causes an error.

The data page (a page restore)

Full, Bulk-logged Recovery نستطيع إستعادة page مضروبة فقط في حالات ال

Under the full recovery model or bulk-logged recovery model, you can restore individual databases. Page restores can be performed on any database, regardless of the number of filegroups.

6.1. Overview of Restore Scenarios

A restore scenario in SQL Server is the process of restoring data from one or more backups and then recovering the database. The supported restore scenarios depend on the recovery model of the database and the edition of SQL Server.

The following table introduces the possible restore scenarios that are supported for different recovery models.

Restore scenario	Under simple recovery model	Under full/bulk-logged recovery models
Complete database restore	سيتم إسترجاع آخر Full backup بالإضافة لآخر Differential في حال وجوده This is the basic restore strategy. A complete database restore might involve simply restoring and recovering a full database backup. Alternatively, a complete database restore might involve restoring a full database backup followed by restoring and recovering a differential backup.	يتم إسترجاع آخر log backup بعد ال D حسب تسلسلهم تماماً حيث نبدأ ب Tail with Norecovery backup لوضع ال DB في وضع offline ثم ننتهي ب Tail Restore With Recovery لنفس ال backup التي قمنا بها لإعادة ال DB لوضع Online This is the basic restore strategy. A complete database restore involves restoring a full database backup and, optionally, a differential backup (if any), followed by restoring all subsequent log backups (in sequence). The complete database restore is finished by recovering the last log backup and also restoring it (RESTORE WITH RECOVERY).
File restore *	لا نستطيع القيام ب restore سوى لملف أو عدة ملفات من نوع ال Read-Only ويجب على الأقل وجود File Group واحدة من نوع Read Only في ال DB Restore one or more damaged read-only files, without restoring the entire database. File restore is available only if the database has at least one read-only filegroup.	نستعيد الملفات ال Read-Only و ملفات ال Read-Write حيث نستطيع إستعادتهم بطريقة Online (يحتاج نسخة Enterprise) أو Offline عند إستخدام طريقة ال Online تبقى ال DB في وضع Online و لكن الملفات المراد إستعادتها توضع في وضع Offline Restores one or more files, without restoring the entire database. File restore can be performed while the database is offline or, for some editions of SQL Server, while the database remains online. During a file restore, the filegroups that contain the files that are being restored are always offline.

Page restore	Not applicable	مدعومة، و بطريقتين Online (تحتاج نسخة Enterprise – و تصبح الصفحة المضروبة بوضع offline ولكن تبقى ال DB في وضع Online) أو طريقة Offline Restores one or more damaged pages. Page restore can be performed while the database is offline or, for some editions of SQL Server, while the database remains online. During a page restore, the pages that are being restored are always offline. An unbroken chain of log backups must be available, up to the current log file, and they must all be applied to bring the page up-to-date with the current log file.
Piecemeal restore *	يتم إستعادة ال DB على مراحل على مستوى ال File Groups حيث يتم اولا إستعادة ال Primary file و جميع ملفات ال Read/Write لنسمح لل users بالعمل (حيث تكون جميع الملفات بوضع Offline), ثم نستعيد باقي ال File Groups (تكون باقي ال File Groups بوضع "Recovery Pending" و ال DB تكون أصبحت Online) و تكون بطريقتين: Online (تكون ال DB بوضع Online ما عدا ال File او ال File Group الذي نقوم بإستعادته), Offline (تصبح كل ال DB بوضع Offline لحظة البدء بإستعادة أي ملف, إلى أن ننتهي من إستعادته) Restore and recover the database in stages at the filegroup level, starting with the primary and all read/write, secondary filegroups.	نفس حالة ال Simple Restore ما عدا أنها لا تقوم بإستعادة ملفات ال Read-Write مع ال Primary File أول الشيء Restore and recover the database in stages at the filegroup level, starting with the primary filegroup.

* Online restore is supported only in the Enterprise edition.

6.2. Steps to restore a database

To perform a file restore (الـ الملف مضروب مثلاً أو محذوف بالغلط), the Database Engine executes two steps:

- Creates any missing database file(s). سيتم إعادة إنشاء للملف أو الملفات بعد التشبيك إذا الملف موجود أو لا
- Copies the data from the backup devices to the database file(s).

ثم يتم نقل الـ Data له و تنفيذ الـ Recovery

- يجب تذكر ما يلي : Restore = Physical = إسترجاع

Recovery = Logical = عملية من الـ System لجعل الـ

Transactions متوافقة مع بعض. حيث عملية الـ Recovery تقوم بها الـ DB لتكون

متوافقة مع بعض

To perform a database restore, the Database Engine executes three

steps:

- يتم التحقق من وجود الـ Primary, Secondary, Log files , حيث يقوم بإعادة إنشاء لهم

في حال عدم وجودهم

- يقوم بنسخ كل الـ data, log, index pages لهم من ملفات الـ backup

- يقوم بعملية الـ Recovery التي يجعل بها الـ Transactions متوافقة مع بعضها (

Consistent)

- ملاحظة:

- Creates the database and transaction log files if they do not already exist.
- Copies all the data, log, and index pages from the backup media of a database to the database files.
- Applies the transaction log in what is known as the recovery process.

Regardless of how data is restored, before a database can be recovered, the SQL Server Database Engine guarantees that the whole database is logically consistent. For example, if you restore a file, you cannot recover it and bring it online until it has been rolled far enough forward to be consistent with the database.

6.2.1. Advantages of a File or Page restore

Restoring and recovering files or pages, instead of the whole database, provides the following advantages:

- Restoring less data reduces the time required to copy and recover it. (RTO)
- On SQL Server restoring files or pages might allow other data in the database to remain online

during the restore operation.

6.2.2. Recovery and the transaction log

عند إنتقال Page إلى ال buffer للبدأ بعملية تعديل عليها يتم التحقق من قيمة ال Checksum الموجودة في ال header الخاص بها
و في حال كانت قيمة ال Checksum المحسوبة غير الموجودة في ال header يعرف بأنها صفحة مضروبة, و يقوم بوضعها في DB معينة إسمها msdb , و نقوم بإستعادتها إذا أردنا لاحقاً.....
و في حال كانت نتيجة ال Checksum متوافقة, يطلق على الصفحة إسم Clean Page
ثم نبدأ بالتعديل عليها و يصبح إسمها Dirty Page و يتم كتابة كل سطر من ال Transaction في ال Log file على شكل أسطر قبل تجميع ال Committed منهم وكتابتهم على ال Data File (عملية ال Checkpoint)
و بالنسبة لل Transactions التي لم يطبق عليها Commit بعد, يطلق عليها إسم Active transaction - AT

For most restore scenarios, it is necessary to apply a transaction log backup and allow the SQL Server Database Engine to run the recovery process for the database to be brought online. Recovery is the process used by SQL Server for each database to start in a transactionally consistent - or clean - state.

In case of a failover or other non-clean shut down, the databases may be left in a state where some modifications were never written from the buffer cache to the data files, and there may be some modifications from incomplete transactions in the data files. When an instance of SQL Server is started, it runs a recovery of each database, which consists of three phases, based on the last database checkpoint:

- **Analysis Phase** analyzes the transaction log to determine what is the last checkpoint, and creates the Dirty Page Table (DPT) and the Active Transaction Table (ATT). The DPT contains records of pages that were dirty at the time the database was shut down. The ATT contains records of transactions that were active at the time the database was not cleanly shut down.

- **Redo Phase**

يقوم ال SQL-Engine بعمل Redo لجميع المناقلات التي تم عمل Commit لها في ال log file ولكن لم تنتقل بعد إلى ال Data file لسبب ما عند حدوث ال shut down او العطل في ال DB – فعليا يقوم ب redo لمحتوى ال Dirty Page Table و يتم الاعتماد على رقم ال MinLSN

rolls forwards every modification recorded in the log that may not have been written to the data files at the time the database was shut down. The minimum log sequence number (minLSN) required for a successful database-wide recovery is found in the DPT, and marks the start of the redo operations needed on all dirty pages. At this phase, the SQLServer Database Engine writes to disk all dirty pages belonging to committed transactions.

- **Undo Phase**

يقوم ال SQL-Engine بعملية Undo للمناقلات التي بحالة Active – فعليا للتي في جدول ال Active Transaction Table

حيث في نسخة ال Enterprise فقط، تعود ال DB لوضع online بعد عملية ال Redo و في باقي النسخ تنتظر للإنتهاء من عملية ال Undo لتعود لوضع Online

rolls back incomplete transactions found in the ATT to make sure the integrity of the database is preserved. After rollback, the database goes online, and no more transaction log backups can be applied to the database.

Information about the progress of each database recovery stage is logged in the SQL Server error log. The database recovery progress can also be tracked using Extended Events.

Viewer

Recovery and the transaction log

- When an instance of SQL Server is started, it runs a recovery of each database, which consists of three phases, based on the last database checkpoint:
- **Analysis Phase** analyzes the transaction log to determine what is the last checkpoint, and creates the Dirty Page Table (DPT) and the Active Transaction Table (ATT). The DPT contains records of pages that were dirty at the time the database was shut down. The ATT contains records of transactions that were active at the time the database was not cleanly shut down.
- **Redo Phase** rolls forwards every modification recorded in the log that may not have been written to the data files at the time the database was shut down. The minimum log sequence number (**minLSN**) required for a successful database-wide recovery is found in the DPT, and marks the start of the redo operations needed on all dirty pages. At this phase, the SQL Server Database Engine writes to disk all dirty pages belonging to committed transactions.
- **Undo Phase** rolls back incomplete transactions found in the ATT to make sure the integrity of the database is preserved. After rollback, the database goes online, and no more transaction log backups can be applied to the database.

Handwritten notes and diagrams:

- Red circles around "Analysis Phase" and "Redo Phase".
- Red arrows pointing from "Analysis Phase" to "Redo Phase" and from "Redo Phase" to "Undo Phase".
- Red boxes around "DPT" and "ATT".
- Red text "P." and "DP" near the top.
- Red text "ATT", "DPT", "LF", "CP", "PF", "R" near the bottom.

6.3. Accelerated database recovery

ميزة تسريع عملية ال Recovery الخاصة بال DB متوفرة فقط في نسخة ال 2019. حيث تصبح السرعة 1.5x و لتفعيلها:
`alter database <db_name> set accelerated_database_recovery = on`

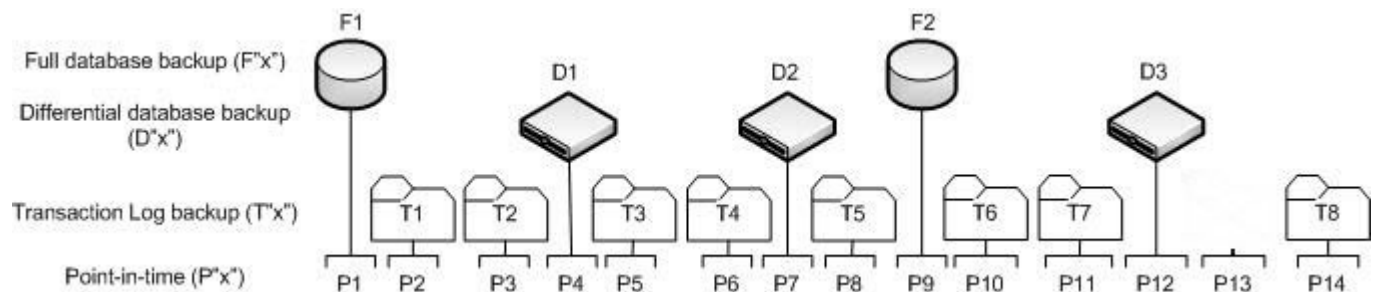
Accelerated database recovery is available in SQL Server 2019 (15.x) and Azure SQL Database. Accelerated database recovery greatly improves database availability, especially in the presence of long-running transactions, by redesigning the SQL Server Database Engine recovery process. A database for which accelerated database recovery was enabled completes the recovery process significantly faster after a failover or other non-clean shut down. When enabled, Accelerated database recovery also completes rollback of canceled long-running transactions significantly faster.

You can enable accelerated database recovery per-database on SQL Server 2019 (15.x) using the following syntax:

```
ALTER DATABASE <db_name> SET ACCELERATED_DATABASE_RECOVERY = ON;
```

7. Restore Scenarios – Full Recovery Model

Let's say that we have this series of backups



7.1. To Restore to point P6:

Restore Latest Full + Latest Differential + All Transaction Log Backups after latest Differential (if exists) or Full, all restores must be in with norecovery except the latest one.

Restore Database iis404 from Disk = 'c:\temp\1.Bak' With File = 1, Norecovery, Nounload, Replace, Stats = 5

Restore Database iis404 from Disk = 'c:\temp\d1.Bak' With File = 1, Norecovery, Nounload, Stats = 5

Restore Log iis404 from Disk = 'c:\temp\t3.Trn' With File = 1, Norecovery, Nounload, Stats = 5

Restore Log iis404 from Disk = 'c:\temp\t4.Trn' With File = 1, Recovery, Nounload, Stats = 5

- **Stats** = x means that SSMS shows the percentage of progress in x% (Optional)
- **Nounload** is a tape option, if you are restoring from tape, specifying this will ensure that the tape is not unloaded from the drive once the restore is complete, if you're not restoring from a tape drive this option is ignored. (Optional)
- **Norecovery** leaves the database offline until the option with recovery set, because users must not access before finishing.

Optionally it's possible to Specify an UNDO File When Restoring Transaction Log T4

Restore Log iis404 from Disk = 'c:\temp\t4.TRN' WITH FILE = 1, STANDBY = 'c:\temp\Rollbackundofile.Tuf', NOUNLOAD, STATS = 5

Restore Database iis404 with Recovery

7.2. To Restore to a point after latest backup:

In reality, damage does not occur at the moment of a backup end, so if the damage occurred after P14 and there were transactions that were committed but not backed up, so first thing we try to create a tail log backup which is a log backup containing the aforementioned transactions and leaves the database in RESTORING mode (no recovery):

Backup Log iis404 to disk = 'c:\temp\T.trn' with norecovery

Restore Database iis404 from Disk = 'c:\temp\1.Bak' with norecovery

Restore Database iis404 from Disk = 'c:\temp\d3.Bak' with norecovery

Restore Log iis404 from Disk = 'c:\temp\t8.Trn' with norecovery

7.3. Restore to a point of time

في حال قمنا بتعديل على ال Database بوقت 14:00, ثم أردنا التراجع عن هذا التعديل..
نقوم بالإستعادة لأخر زمن قبله (حتى لو قبل بثواني قليلة = 13:59:58) بالطريقة التالية :
نقوم بعرض جميع المعلومات قبل تاريخ الحادثة:

```
Select * from <db_name> --13:59:58
```

نقوم ب restore لكل المعلومات التي قبل الحادثة:
أولا ننشأ Tail-Log_backup - نستخدم غير db لكي نستطيع وضع الحالية بوضع offline :

Use master

```
Backup log <db_name> to disk='E:\db_name\name.bak' with norecovery
```

ثم نستعيد آخر full backup للملفات, و من بعدها نستعيد آخر tail backup مع التوقف عند نقطة زمنية محددة:

```
Restore database <db_name> from disk='E:\db_name\full.bak' with norecovery
```

```
Restore log <db_name> from disk='E:.....\name.bak' with norecovery ,
```

```
Stopat = 'apr 26, 2021 13:59:58 pm'
```

Go

ثم نعيد ال Transactions لوضع Online:

```
Restore log <db_name> with recovery
```

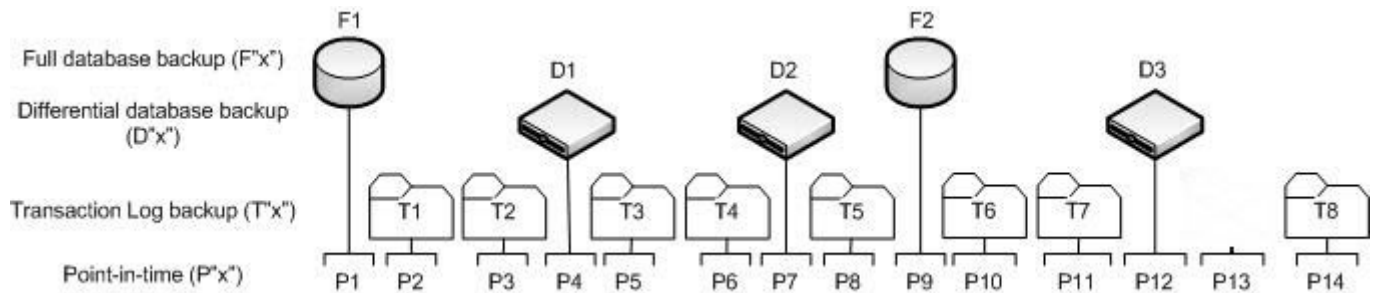
نقوم بالتأكد:

```
Use <db_name>
```

Go

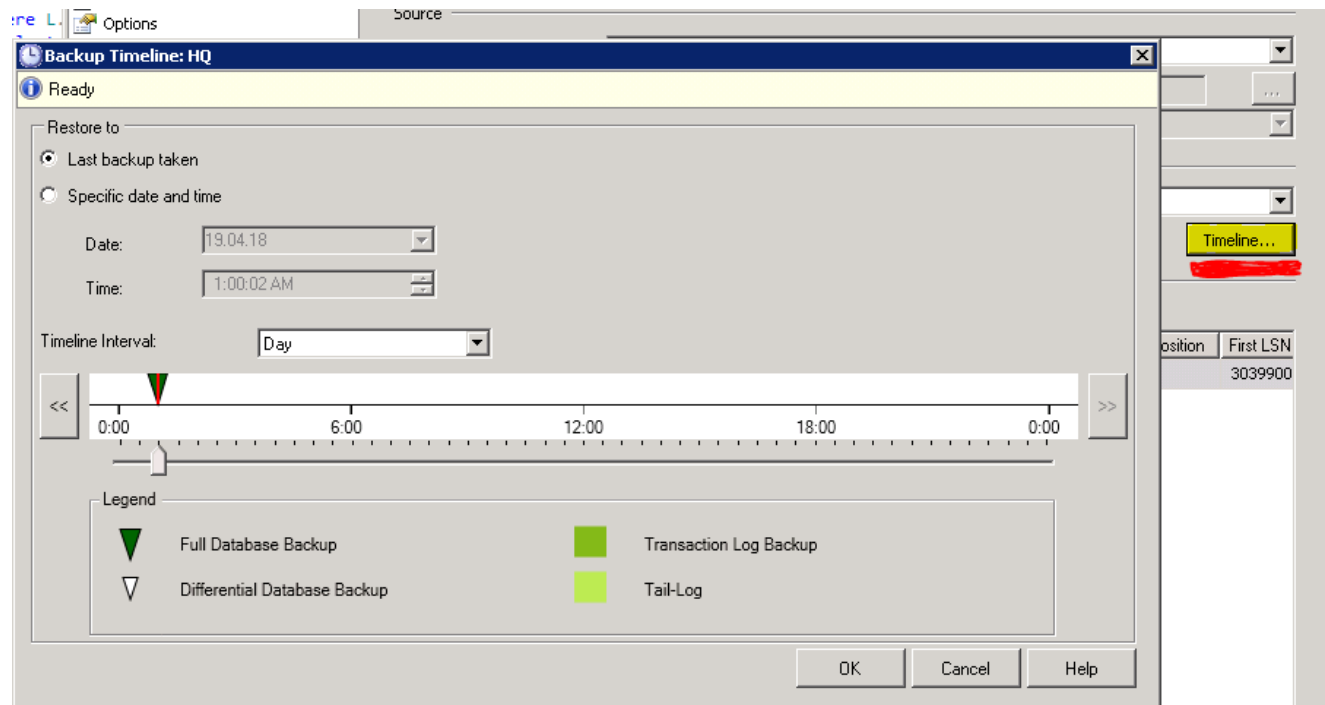
```
Select * from <db_name>
```


If we want to restore to a certain point of time before T4 and after T3



Restore Log iis404 from Disk = 'c:\temp\t4.TRN' WITH FILE = 1, STANDBY = 'c:\temp\Rollbackundo\file.Tuf', NOUNLOAD, STATS = 5, STATS = 10, STOPAT = N'2020-02-13T19:55:33'

We can restore to a point in time by the GUI as well



7.4. Piecemeal Restore

نستعمل معها عبارة "with partial" ضمن تعليمة ال restore

تكون على مراحل, حيث نستعيد أولا ال Primary File و جميع ال Read/Write files في حال ال simple recovery و نستعيد ال Primary file لوحدها في ال full/bulk recovery

ثم نكمل إستعادة باقي ال File groups واحد تلو الآخر

و تكون على نوعين: Online / offline

حيث في نوع ال Online تعود ال DB لوضع online بعد إستعادة ال Primary file (و ال read/write في حال ال simple) و تبقى باقي الملفات offline إلى أن نستعيدهم (نستخدمه في حال الإهتمام بال RTO)

بينما في ال Offline لن تعود ال DB لوضع Online إلى أن ننتهي من إستعادة جميع الملفات (نستخدمه في حال التحوّف من أخطاء معيّنة)

Piecemeal restore allows databases that contain multiple filegroups to be restored and recovered in stages. Piecemeal restore involves a series of restore sequences, starting with the primary filegroup and, in some cases, one or more secondary filegroups. Piecemeal restore maintains checks to ensure that the database will be consistent in the end. After the restore sequence is completed, recovered files, if they are valid and consistent with the database, can be brought online directly.

Piecemeal restore works with all recovery models, but is more flexible for the full and bulk-logged models than for the simple model.

Every piecemeal restore starts with an initial restore sequence called the partial-restore sequence. Minimally, the partial-restore sequence restores and recovers the primary filegroup and, under the simple recovery model, all read/write filegroups. During the piecemeal-restore sequence, the whole database must go offline. Thereafter, the database is online and restored filegroups are available. However, any unrestored filegroups remain offline and are not accessible. Any offline filegroups, however, can be restored and brought online later by a file restore.

Regardless of the recovery model that is used by the database, the partial-restore sequence starts with a RESTORE DATABASE statement that restores a full backup and specifies the PARTIAL option. The PARTIAL option always starts a new piecemeal restore; therefore, you must specify PARTIAL only one time in the initial statement of the partial-restore sequence. When the partial restore sequence finishes and the database is brought online, the state of the remaining files becomes "recovery pending" because their recovery has been postponed.

7.4.1. Piecemeal Restore Scenarios

All editions of SQL Server support offline piecemeal restores. In the Enterprise edition, a piecemeal restore can be either online or offline. The implications of offline and online piecemeal restores are as follows:

Offline piecemeal restore scenario

In an offline piecemeal restore, the database is online after the partial-restore sequence. Filegroups that have not yet been restored remain offline, but they can be restored as you need them after taking the database offline.

Online piecemeal restore scenario

In an online piecemeal restore, after the partial-restore sequence, the database is online, and the primary filegroup and any recovered secondary filegroups are available. Filegroups that have not yet been restored remain offline, but they can be restored as needed while the database remains online.

7.4.2. Piecemeal Restore of Database (Simple Recovery Model)

In this example, database adb is restored to a new computer after a disaster. The database is using the simple recovery model. Before the disaster, all the filegroups are online. Filegroups A and C are read/write, and filegroup B is read-only. Filegroup B became read-only before the most recent partial backup, which contains the primary filegroup and the read/write secondary filegroups, A and C. After filegroup B became read-only, a separate file backup of filegroup B was taken.

Restore Sequences

1. Partial restore of the primary and filegroups A and C.

```
RESTORE DATABASE adb FILEGROUP='A',FILEGROUP='C'  
FROM partial_backup  
WITH PARTIAL, RECOVERY;
```

At this point, the primary and filegroups A and C are online. All files in filegroup B are recovery pending, and the filegroup is offline.

2. Online restore of filegroup B.

```
RESTORE DATABASE adb FILEGROUP='B' FROM backup WITH RECOVERY;
```

All filegroups are now online.

7.5. Piecemeal Restore of Database (Full Recovery Model)

In this example, database adb is restored to a new computer after a disaster. The database is using the full recovery model; therefore, before the restore starts, a tail-log backup must be taken of the database. Before the disaster, all the filegroups are online. Filegroup B is read-only. All of the secondary filegroups must be restored, but they are restored in order of importance: A (highest), C, and lastly B. In this example, there are four log backups, including the tail-log backup.

Tail-Log Backup

Before restoring the database, the database administrator must back up the tail of the log. Because the database is damaged, creating the tail-log backup requires using the NO_TRUNCATE option:

```
BACKUP LOG adb TO tailLogBackup WITH NORECOVERY, NO_TRUNCATE
```

The tail-log backup is the last backup that is applied in the following restore sequences.

7.5.1. Restore Sequences

Note that The syntax for an online restore sequence is the same as for an offline restore sequence.

1. Partial restore of the primary and secondary filegroup A.

```
RESTORE DATABASE adb FILEGROUP='Primary' FROM backup1
WITH PARTIAL, NORECOVERY
RESTORE DATABASE adb FILEGROUP='A' FROM backup2
WITH NORECOVERY
RESTORE LOG adb FROM log_backup3 WITH NORECOVERY
RESTORE LOG adb FROM log_backup4 WITH NORECOVERY
RESTORE LOG adb FROM log_backup5 WITH NORECOVERY
RESTORE LOG adb FROM tailLogBackup WITH RECOVERY
```

2. Online restore of filegroup C.

At this point, the primary filegroup and secondary filegroup A are online. All the files in filegroups B and C are recovery pending, and the filegroups are offline.

Messages from the last RESTORE LOG statement in step 1 indicate that rollback of transactions that involve filegroup C was deferred, because this filegroup is not available. Regular operations can continue, but locks are held by these transactions and log truncation will not occur until the rollback can complete.

In the second restore sequence, the database administrator restores filegroup C

```
RESTORE DATABASE adb FILEGROUP='C' FROM backup2a WITH NORECOVERY
RESTORE LOG adb FROM log_backup3 WITH NORECOVERY
RESTORE LOG adb FROM log_backup4 WITH NORECOVERY
RESTORE LOG adb FROM log_backup5 WITH NORECOVERY
RESTORE LOG adb FROM tailLogBackup WITH RECOVERY
```

At this point the primary and filegroups A and C are online. Files in filegroup B remain recovery pending, with the filegroup offline. Deferred transactions have been resolved, and log truncation occurs.

3. Online restore of filegroup B.

In the third restore sequence, the database administrator restores filegroup B. The backup of filegroup B was taken after the filegroup became read-only; therefore, it does not have to be rolled forward during recovery.

```
RESTORE DATABASE adb FILEGROUP='B' FROM backup2b WITH RECOVERY
```

7.6. Restore Pages

نستعيد file أو File Groups محددة (أحسن لل RTO من ال Full Backup)
 ال Log لا يحتوي على Pages فلا نستطيع إستعادته بواسطة Page Restore
 هناك Pages محددة لا نستطيع إستعادتهم (Full Text Catalog ,
 و ال page رقم 0 من كل Data File حيث تعتبر Boot Page
 و ال page رقم 9 من الملف رقم 1 – ملف ال Primary – و يكون رقمه 1:9 لأنه يحتوي معلومات إقلاع)

- في حال استخدام ال **Bulk-logged Recovery** نضطر لاستخدام خيار **continue-with-error** -

The goal of a page restore is to restore one or more damaged pages without restoring the whole database. Typically, pages that are candidates for restore have been marked as "suspect" because of an error that is encountered when accessing the page. Suspect pages are identified in the suspect_pages table in the msdb database.

A page restore is intended for repairing isolated damaged pages. Restoring and recovering a few individual pages might be faster than a file restore, reducing the amount of data that is offline during a restore operation.

However, if you have to restore more than a few pages in a file, it is generally more efficient to restore the whole file. For example, if lots of pages on a device indicate a pending device failure, consider restoring the file, possibly to another location, and repairing the device.

Furthermore, not all page errors require a restore. A problem can occur in cached data, such as a secondary index, that can be resolved by recalculating the data. For example, if the database administrator drops a secondary index and rebuilds it, the corrupted data, although fixed, is not indicated as such in the `suspect_pages` table.

7.6.1. Limitations

- Page restore applies to SQL Server databases that are using the full or bulk-logged recovery models. Page restore is supported only for read/write filegroups.
- Only database pages can be restored. Page restore cannot be used to restore the following:
 - Transaction log
 - Full-text catalog
 - Page 0 of all data files (the file boot page)
 - Page 1:9 (the database boot page)
- For a database that uses the bulk-logged recovery model, page restore has the following additional conditions:
 - Backing up while filegroup or page data is offline is problematic for bulk-logged data, because the offline data is not recorded in the log. Any offline page can prevent backing up the log. In this cases, consider using DBCC REPAIR, because this might cause less data loss than restoring to the most recent backup.
 - If a log backup of a bulk-logged database encounters a bad page, it fails unless WITH CONTINUE_AFTER_ERROR is specified.
 - Page restore generally does not work with bulk-logged recovery.

A best practice for performing page restore is to set the database to the full recovery model, and try a log backup. If the log backup works, you can continue with the page restore. If the log backup fails,

you either have to lose work since the previous log backup or you have to try running DBCC must be run with the REPAIR_ALLOW_DATA_LOSS option.

7.6.2. Page restore scenarios:

- **Offline page restore**

All editions of SQL Server support restoring pages when the database is offline. In an offline page restore, the database is offline while damaged pages are restored. At the end of the restore sequence, the database comes online.

- **Online page restore**

SQL Server Enterprise edition supports online page restores, though they use offline restore if the database is currently offline. In most cases, a damaged page can be restored while the database remains online, including the filegroup to which a page is being restored. When the primary filegroup is online, even if one or more of its secondary filegroups are offline, page restores are usually performed online. Occasionally, however, a damaged page can require an offline restore. For example, damage to certain critical pages might prevent the database from starting.

مثال:

ننشأ قاعدة بيانات جديدة:

```
Create database <db_name> on
```

```
(name='svu_data' , filename = 'E:.....name.mdf')
```

```
Log on (name='svu_log' , logname= 'E:.....name.ldf')
```

```
Go
```

```
Use <db_name>
```

سنقوم بإنشاء table , بحيث كل قيمة فيه = صفحة واحدة = 8000 bit

ثم سنقوم بتخريب صفحة معينة

```
Create table <page_1>
```

```
( filler char(8000) )
```


Insert into <page_1> values

(replicate ('A', 8000)), (replicate ('B',8000)),(replicate ('C',8000)),(replicate ('D',8000))

Select count (*) from <page_1>

Select * from <db_name>

سيظهر لدينا 4 أسطر كل منها فيه 8000 حرف من الذي حددناه

نقوم بإنشاء Full Backup لنستعيد منها الصفحة المضروبة لاحقاً

Backup database svu to disk = 'E:\.....full.bak'

نقوم بإستعادة أول صفحة من الملف

DBCC ind(svu, table_name, -1)

سنقوم بتخريب صفحة معينة بإستخدام تعليمة DBCC , حيث نقوم بالكتابة بشكل قسري على صفحة معينة من الملف

فيتم الكتابة عليها بدون تغيير ال Checksum الخاص بها – بسبب عدم الكتابة عليها بواسطة ال SQL-Engine

Alter database svu set single_user with rollback immediate

الآن نقوم بتخريب الصفحة رقم 240 مثلاً من الملف

DBCC writepage (svu, 1, 240, 0, 1, 0x41, 1)

DBCC writepage (svu, 1, 240, 1, 1, 0x41, 1)

DBCC writepage (svu, 1, 240, 2, 1, 0x41, 1)

Alter database svu set multi_user

للتحقق :

Select * from table_name

سيظهر لنا خطأ بالتحقق من ال Checksum الخاص بالصفحة 240

أو نتحقق بإستخدام ال DBCC

DBCC checkdb

كما نستطيع العقور عليها في قاعدة البيانات msdb :

```
Select * from msdb.dbo.suspect_pages
```

حيث سيتم عرض الصفحات المشبوهة مع رقمها و اسم ملفها

كمعلومة : نستطيع إضافة معلومات لجدول يحتوي على صفحات مضروبة

نبدأ عملية ال :restore the corrupted page

أولا ننشأ Tail-Logged-backup :

```
Use master
```

```
Backup log svu to disk='E:\....tail.bak' with no_truncate , norecovery
```

ثانيا نستعيد الصفحة المضروبة

```
Restore database svu page='1:240' from disk='E:...full.bak' with norecovery
```

نستعيد ال Tail-Logged_backup

```
Restore database svu from disk='E:....tail.bak' with norecovery
```

ثم نعيد ال DB لوضع online

```
Restore database svu with recovery
```

```
Use svu
```

و للتحقق :

```
Select * from table_name
```

يجب أن تظهر جميع الصفحات من A إلى E

To restore pages using SSMS

1. Connect to the appropriate instance of the SQL Server Database Engine, in Object Explorer, click the server name to expand the server tree.
2. Expand **Databases**. Depending on the database, either select a user database or expand **System Databases**, and then select a system database.
3. Right-click the database, point to **Tasks**, point to **Restore**, and then click **Page**, which opens the **Restore Page** dialog box.

P.S. *Database* selection specifies the database to restore. You can enter a new database or select an existing database from the drop-down list. The list includes all databases on the server, except the system databases **master** and **tempdb**.

Using Transact-SQL

To specify a page in a RESTORE DATABASE statement, you need the file ID of the file containing the page and the page ID of the page. The required syntax is as follows:

```
RESTORE DATABASE <database_name>  
PAGE = '<file: page> [ ,... n ] ' [ ,... n ]  
FROM <backup_device> [ ,... n ] WITH NORECOVERY
```

Example (Transact-SQL)

The following example restores four damaged pages of file B with NORECOVERY. Next, two log backups are applied with NORECOVERY, followed with the tail-log backup, which is restored with RECOVERY. This example performs an online restore. In the example, the file ID of file B is 1, and the page IDs of the damaged pages are 57, 202, 916, and 1016

```
RESTORE DATABASE <database> PAGE='1:57, 1:202, 1:916, 1:1016'  
FROM <file_backup_of_file_B>  
WITH NORECOVERY;  
RESTORE LOG <database> FROM <log_backup>  
WITH NORECOVERY;  
RESTORE LOG <database> FROM <log_backup>  
WITH NORECOVERY;  
BACKUP LOG <database> TO <new_log_backup>;  
RESTORE LOG <database> FROM <new_log_backup> WITH RECOVERY;
```

References

LeBlanc. P, Assaf. W, Jackson. B, Curnutt M; “**SQL Server 2017 Administration Inside Out**”, Microsoft Press (2018).

<https://docs.microsoft.com/en-us/sql/relational-databases/backup-restore/back-up-and-restore-of-sql-server-databases?view=sql-server-2017>

<https://docs.microsoft.com/en-us/sql/relational-databases/backup-restore/restore-and-recovery-overview-sql-server?view=sql-server->

[ver15#:~:text=A%20restore%20scenario%20in%20SQL,the%20edition%20of%20SQL%20Server.](https://docs.microsoft.com/en-us/sql/relational-databases/backup-restore/restore-and-recovery-overview-sql-server?view=sql-server-ver15#:~:text=A%20restore%20scenario%20in%20SQL,the%20edition%20of%20SQL%20Server.)

<https://docs.microsoft.com/en-us/sql/relational-databases/backup-restore/piecemeal-restores-sql-server?view=sql-server-ver15>

<https://docs.microsoft.com/en-us/sql/relational-databases/backup-restore/example-piecemeal-restore-of-database-simple-recovery-model?view=sql-server-ver15>

<https://docs.microsoft.com/en-us/sql/relational-databases/backup-restore/example-piecemeal-restore-of-database-full-recovery-model?view=sql-server-ver15>